Comparative Evaluation of Classification Algorithms for Telecom Customer Churn Prediction

MEHMET KARATEKIN
Department of Computer Engineering
TOBB University of Economics and Technology
Ankara, Turkey
Student ID: 221101031
mkaratekin@etu.edu.tr

***Abstract*—Telecom operators lose revenue when subscribers leave, so knowing in advance who is likely to churn helps them target retention offers. We treat churn prediction as a binary classification task on the public IBM Telco Customer Churn dataset, which has 7043 subscribers, 19 mixed categorical and numeric attributes, and a churn rate of about 27 percent. To keep the comparison fair and avoid leaking information from the test set, we fit all imputation, encoding, and scaling inside the cross validation folds. Five classifiers are compared: a decision tree, Gaussian naive Bayes, k nearest neighbors, a random forest, and gradient boosted trees. Each is tuned by grid search over stratified five fold cross validation and scored on accuracy, precision, recall, F1, and the area under the ROC curve. The two ensembles do best, both reaching an area under the curve of about 0.84, and a McNemar test finds no significant difference between them. Naive Bayes ranks lowest on area under the curve yet gives the highest recall, a clear precision and recall trade off. A separate study of baseline training, class weighting, and synthetic oversampling shows that rebalancing recovers more churners but lowers precision. Feature importance and Shapley values point to contract type, tenure, and the internet and payment attributes as the main churn signals. The results underline the value of careful evaluation over raw accuracy on imbalanced data.**

***Index Terms*—data mining, customer churn, classification, class imbalance, model comparison**

I. Introduction

Retaining an existing customer is usually cheaper than winning a new one, and in saturated markets such as mobile telecommunications the rate at which customers leave, or churn, directly affects revenue [1]. If an operator can spot the subscribers most likely to leave before they go, it can target retention offers at them rather than spend across the whole customer base. Churn prediction is therefore a supervised learning problem: given a customer's account, service, and billing attributes, predict whether that customer will churn.

Many classification algorithms can be applied to this task, from simple probabilistic and distance based models to tree ensembles, and published results often disagree about which is best because they use different datasets, preprocessing, and evaluation metrics [2]. Two mistakes are easy to make. First, churn datasets are imbalanced, since most customers stay, so a model can reach high accuracy while still missing most of the churners; accuracy on its own is a poor measure here [3]. Second, preprocessing steps that are fit on the full dataset before the split leak information from the test set and inflate the reported scores.

This paper addresses both points with a controlled comparison on the IBM Telco Customer Churn dataset. We ask three questions: which of five common classifiers predicts churn best on this data, whether the gap between the strongest models is statistically meaningful rather than noise, and how different ways of handling class imbalance change the balance between precision and recall.

To answer them we build one preprocessing pipeline, fit only on the training data inside cross validation, tune each model with grid search, and score them all with the same metrics. The five classifiers are a decision tree, Gaussian naive Bayes, k nearest neighbors, a random forest, and gradient boosted trees. Beyond a single split we report cross validation stability and a McNemar test between the two best models, and we read feature importance and Shapley values to see what drives the predictions. We then compare baseline training against class weighting and synthetic minority oversampling.

The rest of the paper is organized as follows. Section II reviews related work on churn prediction and classifier comparison. Section III describes the dataset. Section IV presents the preprocessing, models, and evaluation protocol. Section V reports the results, Section VI discusses them and their limitations, and Section VII concludes.

II. Related Work

Churn prediction in telecommunications has been studied for years as a supervised classification problem on historical customer records. Verbeke et al. compared several classifiers on telecom data and argued that models should be judged by their business value, not accuracy alone [1]. More recent work runs standard machine learning pipelines on large telecom datasets; Ahmad et al., for instance, built a churn model on a big data platform and found that tree based ensembles gave the strongest results [4]. Across these studies, ensemble methods usually beat single classifiers on tabular customer data.

The same pattern shows up outside the churn setting. Across many datasets, Fernandez-Delgado et al. found random forests to be among the most accurate general purpose classifiers [2]. That is one reason we include a random forest and gradient boosted trees as strong baselines, next to simpler models such

as naive Bayes and k nearest neighbors that are cheap to train and easy to read.

Churn is a minority outcome, so the imbalanced learning literature matters too. He and Garcia survey the problem and its common remedies, including resampling the training data and reporting metrics such as the area under the ROC curve and F1 rather than raw accuracy [3]. We combine these standard practices on one public dataset: all preprocessing stays inside cross validation to avoid leakage, a McNemar test checks whether the gap between the two best models is statistically significant, and a separate experiment compares class weighting against synthetic oversampling.

III. Dataset

We use the public IBM Telco Customer Churn dataset [5], a common benchmark built from the customers of a fictional telecommunications company. It holds 7043 customer records, each with 21 columns as distributed. We drop customerID, a unique identifier with no predictive value, which leaves 20 columns: 19 predictor attributes and the binary target Churn.

The predictors fall into three groups. Demographic attributes describe the customer: gender, senior citizen status, and whether they have a partner or dependents. Account attributes cover the contract and billing, namely contract type, paperless billing, payment method, tenure in months, monthly charges, and total charges. Service attributes record which products the customer takes, from phone and multiple lines to internet type, online security, online backup, device protection, technical support, and streaming television and movies. Sixteen of the 19 predictors are categorical and three are numeric. Table I lists them by group.

The target is imbalanced. Of the 7043 customers, 1869 (about 26.5 percent) are labeled as churned and the rest are retained. Because of this skew we do not lean on accuracy alone and also report precision, recall, F1, and the area under the ROC curve.

Table II summarizes the numeric attributes. Tenure runs from 0 to 72 months, monthly charges from about 18 to 119, and total charges from about 19 to 8685. Missing values appear only in TotalCharges: 11 records, all new customers with a tenure of zero, are blank. We treat these as missing and impute them during preprocessing instead of dropping the rows.

A first look at the data already hints at what relates to churn. Fig. 1 gives the churn rate within categories of four attributes. Month to month contracts, fiber optic internet, electronic check payment, and customers without technical support all sit well above the overall 26.5 percent rate, while long contracts and customers who do have support sit well below it. Tenure follows the same logic on the numeric side, as churned customers tend to be much newer. These patterns are what the feature importance analysis in Section V later puts on a firmer footing.

The dataset is public and carries no names, contact details, or other direct identifiers, and we remove the single identifier column it does include. It does record demographic attributes such as gender and senior citizen status. Targeting or denying service on the basis of such attributes raises fairness concerns, so a real deployment would need review; here they serve only to study which factors relate to churn on an anonymized research dataset.

TABLE I
Attributes of the Telco Customer Churn dataset

Group (type)	Attributes
Demographic (cat.)	gender, SeniorCitizen, Partner, Dependents
Account (cat.)	Contract, PaperlessBilling, PaymentMethod
Account (num.)	tenure, MonthlyCharges, TotalCharges
Services (cat.)	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Target (bin.)	Churn

TABLE II
Descriptive statistics of the numeric attributes

Attribute	Mean	Std	Min	Max
tenure (months)	32.4	24.6	0	72
MonthlyCharges	64.76	30.09	18.25	118.75
TotalCharges	2283.3	2266.8	18.80	8684.80

IV. Methodology

A. Preprocessing and data splitting

We first hold out a test set. The data is split 80/20 with stratification on the target, so both parts keep the 26.5 percent churn rate, and the test set is never touched during fitting or model selection. Every other choice is made on the training part alone.

Preprocessing lives in a single transformer that is fit inside each cross validation fold, not once on the full data. The reason is leakage: fitting an imputer or a scaler on all rows before the split would let information from the validation and test rows seep into training and inflate the scores. Numeric attributes are median imputed, which fills the 11 blank TotalCharges values, then standardized to zero mean and unit variance. Categorical attributes are imputed with their most frequent value and one hot encoded, and any category unseen at test time maps to all zeros. One hot encoding expands the 16 categorical predictors, so the model input has 46 columns. The distance based and probabilistic models need the standardization, and it does no harm to the tree models, so applying it to everything keeps the comparison fair.

B. Classification models

The five classifiers cover the main families taught in the course. A decision tree splits the feature space recursively to reduce node impurity; it is easy to read but overfits readily, which we curb through its depth and leaf size. Gaussian naive Bayes applies Bayes' rule assuming the features are conditionally independent given the class, with a Gaussian likelihood for the numeric attributes; it trains fast, but that independence assumption does not hold for the correlated

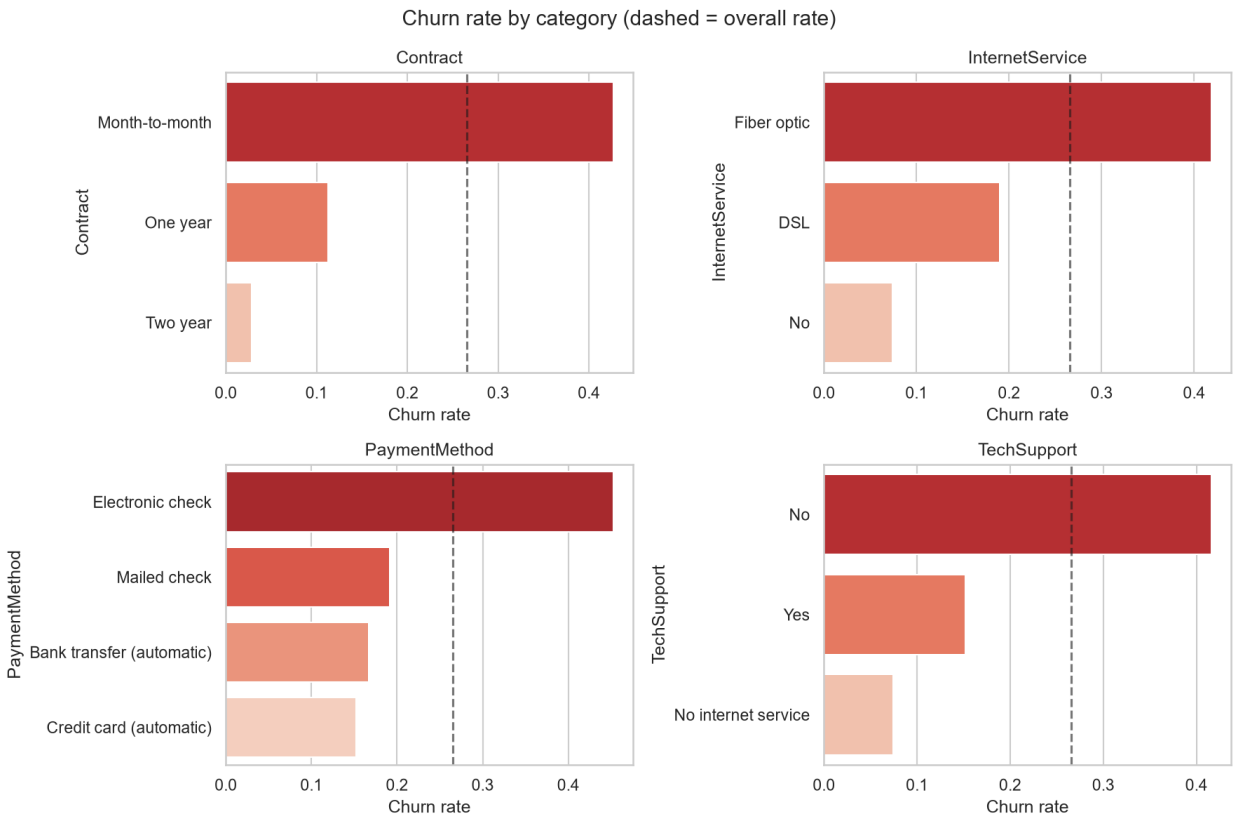

Fig. 1. Churn rate within categories of four attributes. The dashed line marks the overall churn rate of 26.5 percent.

billing columns. The k nearest neighbors classifier labels a customer by a majority vote of its closest training points, which is exactly why the features are standardized first. A random forest averages many decorrelated trees grown on bootstrap samples, cutting the variance of a single tree [6]. Gradient boosted trees, here the XGBoost implementation, add trees in sequence so each one corrects the errors of the current ensemble, with regularization against overfitting [7]. The two ensembles are in because such methods are reliably strong on tabular data; the three simpler models serve as interpretable baselines.

C. Hyperparameter tuning

Each model is tuned with grid search over stratified five fold cross validation on the training set, scored on the area under the ROC curve, which is threshold independent and well suited to imbalanced data. Table III gives the search space for each model. The configuration with the best mean cross validation score is refit on the full training set and then evaluated once on the held out test set.

D. Evaluation metrics and statistical testing

We report accuracy, precision, recall, F1, and the area under the ROC curve. With TP, FP, TN, and FN denoting the confusion matrix counts,

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{1}$$

TABLE III
HYPERPARAMETER GRIDS SEARCHED PER MODEL

Model	Search space
Decision tree	`max_depth` $\in$ {4, 6, 8, None}; `min_samples_leaf` $\in$ {1, 20, 50}
Naive Bayes	`var_smoothing` $\in \{10^{-9}, 10^{-7}, 10^{-5}\}$
k-NN	`n_neighbors` $\in$ {11, 21, 31, 51}; `weights` $\in$ {uniform, distance}
Random forest	`n_estimators` $= 300$; `max_depth` $\in$ {8, 12, None}; `min_samples_leaf` $\in$ {1, 5, 20}
XGBoost	`n_estimators` $= 400$; `max_depth` $\in$ {3, 5, 7}; `learning_rate` $\in$ {0.05, 0.1}; `subsample` $\in$ {0.8, 1.0}

$$F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{2}$$

Precision and recall carry the most weight here, because the churners are the minority class we want to catch without burying the retention team in false alarms. To confirm the ranking is not an artifact of one split, we also report the mean and standard deviation of the cross validation area under the curve for each model. We then compare the two best models with McNemar's test on their test set predictions, using the exact binomial form on the discordant pairs, a standard way to check whether two classifiers differ significantly [10].

E. Handling class imbalance

We also look at how imbalance handling shifts the precision and recall balance, keeping XGBoost as a fixed base model. Three settings are compared: training as is, weighting the positive class by the ratio of negative to positive training examples, and oversampling the minority class with SMOTE [8]. SMOTE runs inside the training folds only, never on the test data, so the evaluation stays honest.

F. Interpretation and implementation

We read two views of what drives the predictions. The first is the impurity based feature importance of the tree ensembles; the second is SHAP values, which attribute each prediction to its features in an additive, locally consistent way [9]. Everything runs in Python: scikit-learn [11] for preprocessing, the classical models, and evaluation, the XGBoost library for gradient boosting, and imbalanced-learn for SMOTE. Every random operation is seeded so the results can be reproduced.

V. Results

A. Model comparison

Table IV reports the five tuned models on the held out test set. The random forest gives the best area under the ROC curve at 0.844, with XGBoost just behind at 0.841. The decision tree and k nearest neighbors come next, and Gaussian naive Bayes has the lowest area under the curve at 0.808. Accuracy agrees at the top but misleads at the bottom: naive Bayes reaches the highest recall of any model, 0.840, yet the lowest accuracy, 0.696. It calls many customers churners, so it catches most of the real ones but raises a lot of false alarms, which drags its precision down to 0.460. The ROC curves in Fig. 4 keep the same ordering, with the two ensembles above the rest across most thresholds. The precision recall curves in Fig. 2 say the same thing and are the sharper view here, since they focus on the minority churn class.

B. Stability and significance

A single split can flatter one model, so Table V lists the mean and standard deviation of the cross validation area under the curve. The standard deviations are small which are between 0.009 and 0.012. The two ensembles lead on both the cross validation and the test set so the ordering is stable. Only the middle pair, the decision tree and k-NN, swap places and by a margin far smaller than the fold to fold variation. Only 0.003 area under the curve separates the two ensembles, which is well inside that spread. The McNemar test pins this down: on the test set the random forest is right where XGBoost is wrong on 42 cases, and the reverse on 34, giving $p = 0.42$. We cannot claim the random forest is really better than XGBoost on this data; the two are effectively tied.

C. Where the models differ

The confusion matrices in Fig. 5 set the random forest against naive Bayes. The random forest keeps false positives low but misses about half of the churners, the low recall already seen in Table IV. Naive Bayes runs the other way,

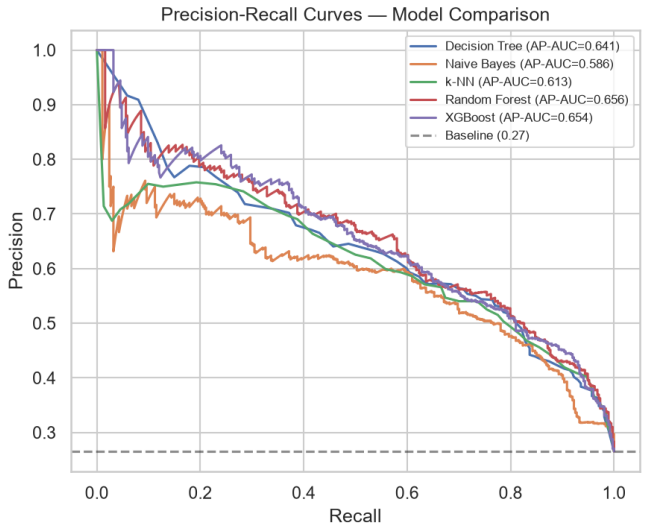

Fig. 2. Precision-recall curves on the test set for the five tuned models.

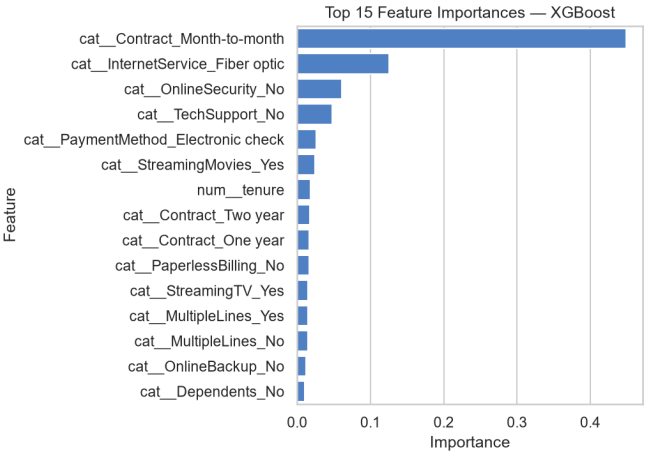

Fig. 3. Top feature importances from the gradient boosted trees.

catching most churners but also flagging many loyal customers. Neither is automatically the better choice; it depends on how costly a missed churner is next to a wasted retention contact.

D. What drives the predictions

The impurity based importances of the two ensembles and the SHAP summary in Fig. 6 agree on the leading factors. A month to month contract, short tenure, fiber optic internet, no online security or technical support, and electronic check payment all push the predicted churn probability up, while two year contracts and long tenure pull it down. These are the same segments that stood out in the exploratory analysis, which is a sign the models are reading sensible signals rather than noise. Fig. 3 shows the gradient boosted tree importances, which put the same contract, tenure, and internet attributes on top.

E. Effect of imbalance handling

Table VI compares three ways of handling the imbalance, all on the same untuned XGBoost base model so that only the balancing method changes; this is why its scores sit a little below the tuned XGBoost in Table IV. Class weighting and SMOTE both raise recall over the baseline, from 0.527 to 0.591 and 0.546, but they cost precision, so F1 hardly moves. The area under the curve stays almost flat, as expected for a measure that ignores the decision threshold. Rebalancing mostly shifts the operating point toward catching more churners, and whether the extra false alarms are worth it is a business call, not a modeling one.

TABLE IV
TEST SET PERFORMANCE OF THE FIVE TUNED MODELS, SORTED BY AREA UNDER THE ROC CURVE

Model	Acc.	Prec.	Rec.	F1	AUC
Random forest	0.803	0.671	0.503	0.575	**0.844**
XGBoost	0.797	0.646	0.521	0.577	0.841
Decision tree	0.794	0.640	0.508	0.566	0.831
k-NN	0.784	0.593	0.588	0.591	0.830
Naive Bayes	0.696	0.460	**0.840**	0.594	0.808

TABLE V
CROSS VALIDATION AREA UNDER THE CURVE (MEAN, STANDARD DEVIATION) AND TEST AREA UNDER THE CURVE

Model	CV AUC mean	CV AUC std	Test AUC
Random forest	0.848	0.009	0.844
XGBoost	0.845	0.011	0.841
k-NN	0.835	0.011	0.830
Decision tree	0.831	0.012	0.831
Naive Bayes	0.820	0.012	0.808

TABLE VI
EFFECT OF IMBALANCE HANDLING ON A FIXED XGBOOST BASE MODEL

Setting	Acc.	Prec.	Rec.	F1	AUC
Baseline	0.774	0.581	0.527	0.553	0.807
Class weight	0.762	0.548	0.591	0.569	0.804
SMOTE	0.768	0.565	0.546	0.555	0.800

VI. DISCUSSION

The comparison gives a clear but nuanced answer to our first question. The two tree ensembles are the strongest models by area under the ROC curve, and they sit so close that a McNemar test cannot separate them, so the honest reading is that random forest and XGBoost are tied rather than that one wins. That is worth saying plainly, because a 0.003 gap in a single table would otherwise invite an overclaim. The decision tree and k nearest neighbors sit a step below, and Gaussian naive Bayes comes last on ranking quality. Its weakness is no surprise: several attributes are strongly related, such as monthly charges, total charges, and tenure, which breaks the conditional independence that naive Bayes assumes.

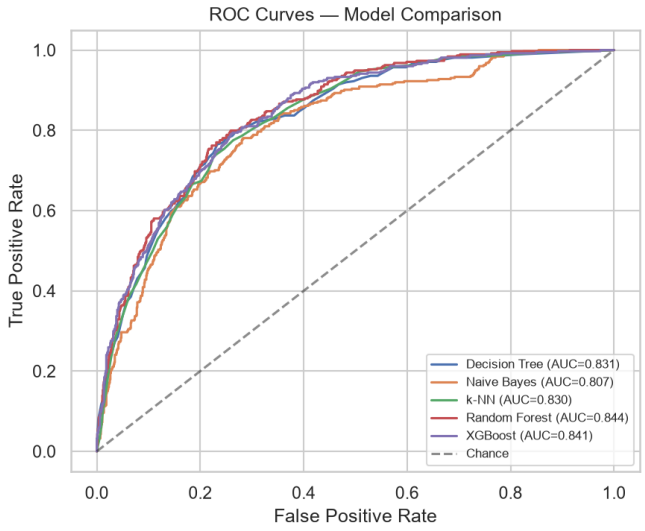

Fig. 4. ROC curves on the test set for the five tuned models.

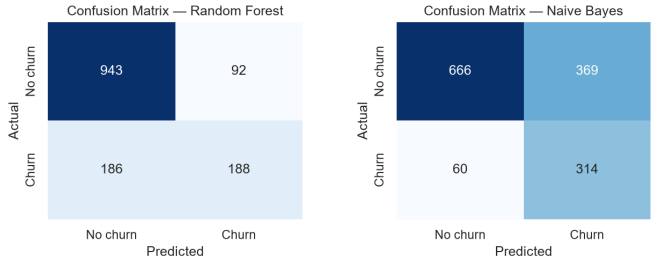

Fig. 5. Test set confusion matrices for the random forest (left) and Gaussian naive Bayes (right).

What the numbers teach is less about the ranking and more about the contrast between naive Bayes and the ensembles. Naive Bayes has the lowest accuracy yet the highest recall, because it predicts churn for a lot of customers. When a missed churner costs more than a wasted retention call, that can be the better behavior, even against a more accurate model that stays quiet. This is why we did not rank the models on accuracy and looked at the full confusion matrices and the precision recall curves instead.

The imbalance experiment tells the same story. Class weighting and SMOTE do not make the model fundamentally better; they shift its operating point toward higher recall at the cost of precision, while the area under the curve barely moves. Choosing between them comes down to the cost the operator puts on each kind of error, not to model quality.

A few limitations qualify all of this. The results come from one public dataset for one fictional company, so they need not carry over to other markets. The headline numbers rest on a single train and test split, though the small cross validation spread suggests the ranking is stable. We did not calibrate the predicted probabilities, so they are better read as rankings than as true churn likelihoods. The feature importance and

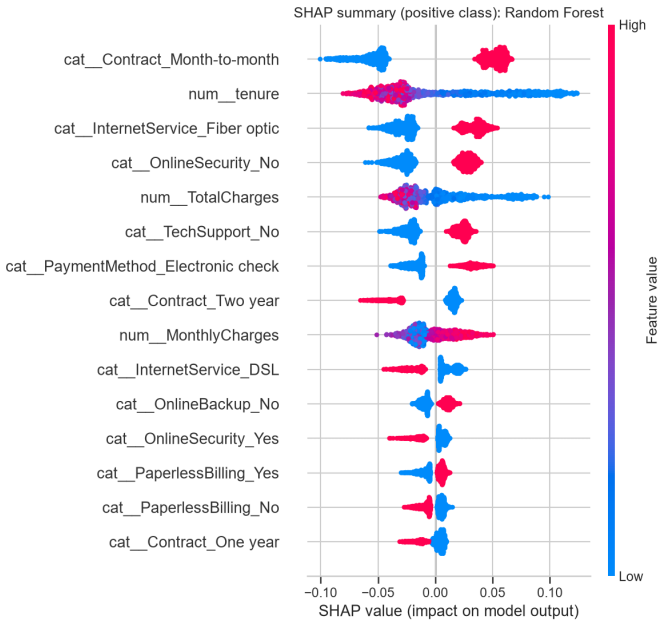

Fig. 6. SHAP summary for the best model, ranking features by their impact on the predicted churn probability.

SHAP analyses describe association, not causation, so a highly ranked attribute is a signal of churn rather than a proven cause. SMOTE can also invent synthetic points that mix categorical values in unrealistic ways. And the data carries demographic attributes such as gender and senior citizen status; using them to drive retention or pricing would need a fairness review before any real deployment.

VII. Conclusion and Future Work

We compared five classifiers on the Telco Customer Churn dataset under one leakage free pipeline, with tuning, cross validation, and a statistical test. The two ensembles reach the best area under the curve, about 0.84, and come out statistically tied, while naive Bayes gives up accuracy for the highest recall. Handling the class imbalance pushes the balance toward catching more churners but leaves the ranking quality unchanged. Contract type, tenure, and the internet and payment attributes are the strongest churn signals in every model.

What we take from this is that on imbalanced data the evaluation protocol counts for as much as the algorithm. Accuracy alone would have masked how naive Bayes behaves, and a single split would have let us claim, wrongly, that one ensemble beats the other.

There is room to go further. The decision threshold could come from an explicit cost model rather than the default, the predicted probabilities could be calibrated, and cost sensitive training or a stacked ensemble is worth a try. Running the same pipeline on other churn datasets, and auditing the demographic attributes for fairness, would show how far these conclusions travel.

Code and Data Availability

The code and dataset, with instructions for reproducing every result and figure, are available at https://github.com/mehmet5038/churn-classifier-benchmark.

AI Assistance Declaration

AI-assisted tools were used during this project. An AI assistant helped write and debug the analysis code, generate the figures, and draft and format parts of this report in LaTeX. The author chose the topic, dataset, and methods, interpreted the results, and decided what to report, checking every reported number against the program output and reviewing and editing the text before including it. The author takes full responsibility for the accuracy, originality, and integrity of the submitted work. A fuller account of how these tools were used is given in the separate Personal Reflection and AI Use Disclosure Form.

References

[1] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[2] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?," *Journal of Machine Learning Research*, vol. 15, pp. 3133–3181, 2014.

[3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[4] A. K. Ahmad, A. Jafar, and K. Aljoumaa, "Customer churn prediction in telecom using machine learning in big data platform," *Journal of Big Data*, vol. 6, no. 28, 2019.

[5] IBM Sample Data Sets, "Telco Customer Churn," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[10] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.